

Rasqal: A Solver-inspired Quantum-Classical Runtime

John Dumbell

January 2025

Abstract

Rasqal is a solver-inspired classical-quantum hybrid runtime that uses combined classical-quantum contexts to optimize, simulate, and split hybrid algorithms as they execute. It inherits many ideas from interpreted languages such as Python, runtimes like the CLR, and formal methods-driven static analysis tooling. Current capabilities allow it to run full profile QIR, executing or squashing arbitrarily complex and interwoven classical-quantum data- and control-flow, with only some exceptions.

This paper introduces all of Rasqals core concepts and structures, including the symbolic execution engine, its central data structures, and details such as how it infers classical-quantum boundaries and decide when to heavily defer or merge circuit execution. We also cover the exceptions when it can't lower and execute operations, future directions of experimentation and how such dynamic systems can fit into the overall hybrid compilation and execution pipeline.

Contents

1	Introduction	1
2	Architecture	2
2.1	Overview	2
2.2	Parsing	2
2.3	Logic Graph	3
2.4	Symbolic Runtime	4
2.4.1	Multi-Backend Runtime Context	4
2.4.2	Dynamic Graph Evolution	4
2.4.3	Implicit QPU Liveness Scoping	6
2.4.4	Quantum Circuit Deferral	7
2.4.5	Quantum Expression Merging	7
2.4.6	Circuit Synthesis Module	8
2.5	Python Bindings	9
3	Conclusion	9
3.1	Acknowledgments	9
3.2	References	9

1 Introduction

Current QPUs are still in their nascent stages, usable for experimentation but are at qubit sizes and accuracy that can't, currently, run key algorithms that the community want to use to prove definitive quantum advantage.

While improving hardware is one way to increase these capabilities to run such algorithms, another is to reduce the amount of work passed to the QPU itself via novel compilation and runtime approaches.

This paper introduces a tool called Rasqal that can be to facilitate the latter. Its approach is to focus on heavy analysis of a hybrid algorithms' classical and quantum logic, execute the logic locally and then synthesize the smallest quantum circuits it can to complete any quantum calculations.

In comparison, the majority of other hybrid compilers and runtimes work to isolate quantum context from classical, creating distinct barriers between the two and lowering or hoisting operations as necessary. The end result is a purely-quantum segment of logic embedded within a surrounding classical one. This

classical logic can do numerous things which include QEC application, post/pre-processing or run-until-complete hoisted iterations. However, the focus is on creating a distinction between the classical and quantum aspects, then optimizing and executing each of them separately.

Rasqal executes both quantum and classical logic side-by-side and feeds their current context into each other to provide novel opportunities for optimization, circuit splitting/knitting, and execution as a whole. Its stated design goals are simple: if you execute and process hybrid algorithms in an interpreted manner, going line-by-line and using intermediate results from both classical and quantum processing, what sort of novel optimizations and rewriting can you do? Can you split large circuits into smaller chunks that return intermediate results that are then used to drastically reduce further processing? If you heavily defer a quantum circuit's execution and take into account how it is used in classical logic, are you able to run a much simpler circuit to get the desired result?

While these questions aren't fully answered in this paper, it outlines the current progress towards those answers, the data structures and algorithms used to power Rasqal, and future directions of research.

The paper is laid out thusly: the first section will introduce all important modules of Rasqal, its algorithms, concepts and data structures. It's ordered front-to-back, starting with the IR parser, following through to the symbolic executor and circuit synthesizer, and finishing with the Python bindings which can be used to run and retrieve results. Following from that are example use-cases where such a tool could fit into existing pipelines and ideas, further work and direction of development, rounding off with a conclusion and acknowledgements.

2 Architecture

2.1 Overview

Rasqal is written in Rust with Python bindings and interfaces to its internal structures. It takes in hybrid IRs, such as QIR, then executes them directly. Currently it only takes in QIR, but there is no reliance upon it or LLVM to power any of its internals, and it also accepts full specification QIR, fully interwoven LLVM and QIR instructions with minimal restrictions applied. This sets it apart from current runtimes and compilers which only accept varied QIR profiles or QIR which only adheres to a particular underlying QPUs capabilities. Rasqal accepts everything and during runtime synthesizes its circuits in a way that can then be run against available and appropriate machines, and exceptions if this is impossible.

There is some important nuance to this synthesis though. Rasqal is built to full capture all aspects of an algorithm within its data structures, with no calls outside of its process which it hasn't already verified as acceptable or stubbed. This means that almost all system calls, such as I/O operations, will be ignored. This was done to make sure the program context given to the symbolic executor is complete, with the only measures of nondeterminism being well-defined arguments fed into an algorithm, and the resultant calls from any QPU. This decision drives many of the design decisions made and powers some of the tools more interesting features.

The following sections describe the important aspects of its design from front-to-back. Starting at QIR parsing, building its graphs, executing them with the symbolic executor, building a circuit with the circuit synthesizer, selecting and calling an appropriate QPU, and finally returning any required result via its Python bindings.

2.2 Parsing

We initially looked at whether it was viable to use LLVMs built-in interpreter or JIT capabilities to parse the QIR, or any other LLVM IR subsets, directly into Rasqal's own data structures. This was not possible due to two reasons: firstly as LLVM is built in C++ options for comprehensive extensibility were limited without building our own C++ module; secondly, the default implementation that was easily accessible via the C API's automatically resolved branching and control-flow. Usually this would be a good thing as it leaves implementers to focus on their own additions to the IR, but for our use-case we needed all that control-flow and data-flow information.

Once parsing has started the passed-in IR gets walked instruction by instruction to see if there is an appropriate graph node or edge that relates. Many operations, such as arithmetic, binary operations and control-flow all map directly into graph nodes or edge control. Others, such as QIR methods, system calls and other intrinsics get checked to see if their operation is allowed, then an appropriate node is added to represent them. Such mappings are not always straight forward, as intrinsics may be performed by a set pattern of nodes and relations, or new graphs. If an intrinsic asks for external data that is not a QPU

operation, such as network or I/O access, it will simply be ignored. In special situations where such data can be stubbed, it will be.

Any memory access is returned to their higher abstract forms of assignments, arrays and variables. Function calls are just turned into nested graphs with well-known arguments, state and outputs which are shared across all invocations.

As it's enforced that the only nondeterministic values are algorithm arguments and QPU measure instructions, the parser itself performs obviously useful constant folding as it goes, leaving more complex analysis to later passes. Currently such passes have not been implemented due to time, and the simplicity of most algorithms mean the majority of cases where folding has impact is already caught and processed. In time more through analysis and contraction of a parsed graph will happen, but for now that is minimal.

After parsing we now have an entry-point graph, a collection of nested graphs, and the required arguments for the algorithm.

2.3 Logic Graph

The graphs generated from the parsing phase are each a directed-acyclic graph (DAG), which internally is called a logic graph, and shares many similarities with traditional control-flow graphs (CFG). Each node is an individual operation while the edges relate to flow of execution. Multiple edges can flow out of a node, but the condition that causes execution to take that edge is embedded as an equality expression on the edge itself, with only one unconditional edge being allowed for both incoming and outgoing flow.

Where it starts to differ more drastically is in the nodes themselves. They represent the algorithm purely through expressions: arithmetic, equality comparisons, bit manipulation, and quantum operations. Most of the control-flow or computational elements such as returns, variable assignments/declarations, branching, are all embedded within the graphs structure or as a part of the contextual state as the graph is executed. This was done to allow for future development towards algorithmic pattern analysis, since both classical and quantum logic has been shrunk to its most simple form.

The available quantum operations are a selection of commonly-used gates and execution metadata, such as explicit qubit activation/deactivation, measure, reset and initialization.

Values are also embedded in the graph at varying points such as equality expressions on an edge and are restricted to certain fundamental types. For classical values: primitives, a reference to another value, or an array. Quantum values only come in two forms: an actual result obtained from a QPU, either full or partial; a deferred or still-being-built circuit that, so far, hasn't been asked by the system to return a result. Some of these values, primarily classical ones, exist in the graph before execution, but are also fundamental for data-flow during execution.

In addition to the above there are a selection of structural nodes which help act as markers or assist in execution but have minimal or no relation to the algorithms calculation. Labels and message logging are two examples of such. Such nodes will come and go as development continues, but the concept of a node which has no relation to the algorithms calculation and is opaque to execution will persist.

As the focus of initial development was on the front-to-back pipeline the graph itself is only in a minimal form to be developed upon later. For example, right now quantum operations are direct mapping to gates, where-as in the future we want to try and represent patterns of gates as their intended operation, rather than precise gates. This will allow for easier comparisons between graphs and open up some fascinating opportunities for optimization and graph re-building.

Here is the full list of nodes available in a logic graph at the time of writing:

- Assignments.
- Arithmetic and bitwise operations.
- Nested graph call.
- Equality comparisons.
- Quantum operations.
 - Initialise.
 - Activate/deactivate qubit.
 - Gates.
 - Measure.

- Reset.
- Labels.
- Returns.
- Logging.
- Throws.

2.4 Symbolic Runtime

The heart of Rasqal is its symbolic execution engine which steps through a logic graphs nodes, taking appropriate edges, and performing all classical, or pseudo-classical operations in-memory and storing the state before moving to the next node. All this information is then used to guide circuit synthesis when quantum operations are found and processed.

Execution starts at the designated entry-point graph which holds the entry-point method for the algorithm, along with arguments that were passed to execute it with. In Rasqal you can build up an algorithm and continuously call it with different arguments. Graphs may evolve during execution, but are reset to their original form for each set of arguments.

From there, it inserts the arguments into the starting execution context, picks the first node in the entry-graph, then symbolically steps through the graph, evaluating all expressions and following edges, until it reaches the end of the entry-point graph and returns results, if any.

There are a three primary modules which power Rasqals primary features: its multi-runtime context which allows flow from CPU, QPU, and in the future GPU results seamlessly into and out of the current symbolic state; its *quantum projections*, which encapsulate circuit synthesis, quantum optimization, quantum bounds scoping and quantum deferred executions; and finally the dynamic graph analysis and overlaying, which allows for classical operations to be dynamically hoisted into memory or lowered into QPU instructions.

We'll now cover each of these in more depth.

2.4.1 Multi-Backend Runtime Context

One of Rasqals main philosophies is that to be able to optimize the quantum circuits to the utmost - including if applicable, not running them at all - that the exact state of an algorithm is known at each step of execution, including prospective quantum state results.

This means during execution all results, or their predicted values, are stored in the context during runtime and can be used to optimize control-flow or calculations based upon them. Classical expressions which are trivial to evaluate can be done immediately. Quantum ones, since the circuits are synthesized piece-meal, require prediction or targeted simulation if such a mid-circuit result is required for the system to make a decision. In some cases such a decision cannot be reached, and if so optimization cannot be performed.

But at each step the results from all available hardware flow into the same state and are used to make better decisions about execution flow, and each other.

2.4.2 Dynamic Graph Evolution

During execution Rasqal looks ahead and behind of the currently-executing step to see if context hints can be used to transform the graph temporarily or permanently.

One of these transformations involves lowering logical conditions directly into a QPU, or alternatively pre-calculating that value and removing the condition entirely. As the runtime is stepping through the graphs, if it reaches a conditional statement it evaluates both sides of the equality check to see if any transformations can be done. If it is entirely classical, the expression is evaluated and branch chosen based on the result. If one of the sides of the conditional is a quantum value, passed on from a measure statement, the system verifies if the body of the conditional can be lowered wholesale into the QPU, and if so, flattens the whole conditional and branch into the QPU itself.

We're going to run through a psudeo-code example of a mid-circuit measurement conditional being detected and lowered to show the step-by-step transformation.

First example is a mid-circuit measurement which triggers further gates. Surrounding code has been omitted for succincity.

```

q1result = measure q1;
if q1result == 1 {
    x q2;
    y q2;
}

```

This can be lowered trivially if the QPU supports it as the body of the conditional is purely quantum with only gate operations with static qubit arguments. The graph itself will not change here, but both condition and body, instead of being processed normally, will be marked for automatic lowering.

A slightly more advanced example is when the conditional body is not, traditionally, purely quantum.

```

# This is an argument passed in from outside the code block. It's value is known by the system, but
arg = ...

q1result = measure q1;
if q1result == 1 {
    rx pi/2 * arg q2;
    ry pi/4 * arg q2;
}

```

In this example the two gates within the conditional body have arguments that are calculated from a value passed in from outside the code block, or even outside the program itself. Traditionally this would be difficult to statically compile away, but as Rasqal knows what 'arg' is at runtime, it can perform the multiplication in-line and pass a known value as the gates argument.

If we were to say 'arg' is 5, then this is what the code simplifies away too.

```

arg = ...

q1result = measure q1;
if q1result == 1 {
    rx 24.6740... q2;
    ry 3.92699... q2;
}

```

There is one more example to highlight that adds in one more complication: nested quantum circuits.

```

# This is an argument passed in from outside the code block. We don't know
# it's value.
qubit_count = ...

# This calls a QPU to generate a random number.
arg = generate_random_number_quantumly(qubit_count)

q1result = measure q1;
if q1result == 1 {
    rx pi/2 * arg q2;
    ry pi/4 * arg q2;
}

```

In this example we've replaced 'arg' with a function call to a quantum circuit which now generates it. Previously Rasqal knew precisely what the value of 'arg' was, so could perform the multiplication trivially. We now no longer have such a guarantee, as each call will return a different number, has itself a variable argument, and requires a call to external hardware to calculate.

Since that is the case, Rasqal will immediately call out to the external hardware when it's deemed appropriate, and return the result. How it deems that such a result can be called, and making sure two isolated circuits don't get merged, will be covered in detail in future sections. But for this example, it deems the function isolated enough to immediately call the QPU and return its actual value. So 'generate_random_number_quantumly' will execute immediately, return its result, and embed that into the current state.

Now that Rasqal knows what 'arg' is in this instance, it follows the same process as before and can perform the multiplication in-line.

All three examples have shown ways in which Rasqal uses the exact information about scope and context to make decisions about control-flow and lowering. Currently most evolutions are ephemeral and do not affect the graph directly, instead changing how the surrounding systems process it. In the future graphs may be changed more permanently, if such a change is evaluated as universally applicable.

2.4.3 Implicit QPU Liveness Scoping

Because quantum circuits are written and run as a batch operation they are different from CPU instructions which can, mostly, be run in sequence. When you see a measure operation you cannot simply perform a measure with the circuit built so far, because you cannot know if that is the end of the circuit.

Many languages and IRs have explicit mechanisms to say when a circuit is finished. QASM does so by reaching the end of the file. Q by having qubit scoping blocks, which when outside means that qubit is no longer used.

If an IR has boundaries defined in the metadata, they are also included in the graph to provide concrete information about when qubits are meant to fall out of scope. When all qubits are dropped from the scope, nothing else can be added to the circuit so it is implied to be complete.

But such boundaries are not required, as Rasqal only considers a circuit complete when its measurement results are used in a classical expression. When this happens it's implied that since the results of a quantum execution are being used in a way that is impossible to lower or optimize any further, then it has to be complete.

We can showcase this in more detail re-using a previous code example.

```
# This calls a QPU to generate a random number.
arg = generate_random_number_quantumly(...)

q1result = measure q1;
if q1result == 1 {
    rx pi/2 * arg q2;
    ry pi/4 * arg q2;
}
```

In this example we previously said that the result of 'arg' is immediately returned. We can now be a bit more truthful, that is only true when 'arg' is used in the multiplication for the RX gate. Until that point the circuit is not marked as complete, and the QPU has not been called. Once that expression is reached the built-up circuit is optimized, targeted and then sent to a QPU with its results being stored in the state so to not trigger any further executions.

A quantum result is linked to its associated quantum projection, which holds cached QPU results, circuit synthesizer, QPU selector, and more. Projections also take care of qubit liveness states and details, where if the graph has metadata blocks about qubit lifetimes, this will be directly used by a projection to help guide its own systems.

A projection is meant to cover a singular QPU execution. Multiple projections can be live at once, covering different parts of the same algorithm across different machines or qubits. Projections get deleted when all their used qubits are no longer live, any associated quantum results have fallen out of scope, and they have no relation to another, currently-live projection that is using their data to optimize itself. In the latter scenario, where a projections direct use has come to an end but it is still being used to tangentially optimize other calls, it is marked as frozen. This means it can no longer be modified and all data it holds is considered static.

2.4.4 Quantum Circuit Deferral

Due to circuits being built incrementally and only being executed when a circuit has been completed, deferring execution until that point is crucial. The deferral mechanism Rasqal employs is straight forward: as the symbolic executor walks the graph if it finds a quantum operation it adds that to the currently active projection, or creates one if it doesn't exist. If a measure operation is found, the projection returns a deferred quantum result linked to that projection and the qubits the measure was applied too. These deferred values then get carried forward in the state until they reach a definitive execution point, a point where they can no longer be deferred.

Currently this happens when any expression requires the deferred value to perform its own operation. This includes arithmetic operations, indexing and equality checks. But this rule is also not definitive, as the system will attempt *lowering* and *expression merging* before it finally decides to execute a deferred value. Examples of lowering was brought up in a previous section, where a measurement result was fed to a conditional, but instead of executing we lowered the entire block into the machine instead. Expression merging will be covered in the next section in more detail, but is the act of continuing the deferral by merging the incoming expression into the deferred execution itself.

There is no limit to how long something can be deferred via expression merging or other transformations, as long as the system has deemed the value not required to be actively executed. This can result in some executions being wholly deferred until the entry-point graph reaches its conclusion. If a deferred value is then returned from the symbolic executor it will then be executed with no further delays.

While further advancements to the expression merging module may heavily complicate the execution pathway for deferred values, in its current state the system executes most deferred values when it has detected the circuit will no longer be added too or it has reached a definitive execution point. This is because circuit brevity is key, so specific merging operations have been proven to be universally beneficial, they won't be applied by default.

2.4.5 Quantum Expression Merging

Quantum expression merging takes the classical concept of expression folding and applies it to our deferred quantum results. This means that when a deferred quantum result meets another expression, if that expressions operation can be merged, or folded, into the deferred result as an addendum it will be. This means as deferred results flow through the system they get incrementally larger, but it also provides more context about *how* such a result will be used, which can then be used to better optimize when the deferred value is actually executed.

Examples are the best way to showcase how this works.

```
result = list()

# Generate 20 random numbers via our QPU, then generate some random seed
# values.
for 20 iterations:
    random_number = quantum_rng()
    first_seed = generate_quantum_seed(random_number);
    second_seed = (first_seed * random_number) + 42;
    final_seed = generate_quantum_seed(second_seed);
    result.add(final_seed);
```

This code has arbitrarily interwoven quantum and classical data-flow each feeding into the other in non-trivial ways: multiple quantum results feeding into separate quantum methods; quantum results being operated on with classical values before being used as an argument to another quantum method; results being added to a list that is outside the loop.

Expression merging could be very useful here as we have two quantum execution blocks linked by a very small bit of classical computation. We'll run through how this would work step-by-step.

```
random_number = quantum_rng()
first_seed = generate_quantum_seed(random_number);
second_seed = (first_seed * random_number) + 42;
```

In this fragment we have two potential targets for merging: `random_number` and `second_seed`. As `random_number` is immediately used as an argument into another quantum circuit this could be a potential target for merging. Although there would have to be an analysis that verified that the merged circuit was both logically comparable and also more effective than both being run separately. This is non-trivial but potentially doable, but for this example we're going to say that no, the system detects this is not mergable and so keeps the calls separate.

This means that `random_number` is now a known value and can be directly injected into the final line, meaning that the results of `second_seed` now becomes the result of the quantum call operated on by two constants. As there are quantum circuits for both multiplication and addition, this could be a very good target for merging onto a deferred quantum result.

Practically in the system when this line is processed the result from `generate_quantum_seed` is still deferred and not been called. When it comes up against our new expression, which for simplicities sake it marks as mergable, it just creates quantum circuits that perform these two arithmetic operations to the current result end of the deferred circuit.

The deferred quantum result still hasn't been executed, but it has absorbed further calculations that have been detected as viable to run in our quantum circuit. This also means that optimizers have access to more contextual and logical information about the circuit to do a better job. This is not guaranteed to always produce better results, but there are opportunities to optimize away large swathes of circuits that, due to the additional context, may not be required.

With this merging complete, let's modify the code example to show what is now happening.

```
result = list()

# Generate 20 random numbers via our QPU, then generate some random seed
# values.
for 20 iterations:
    random_number = quantum_rng()
    first_seed = generate_quantum_seed(random_number);
    final_seed = generate_quantum_seed(first_seed);
    result.add(final_seed);
```

With the removal of the classical expressions everything within the loop body, aside from the list addition, is now a quantum call. Let's get ambitious and now say that we can lower the 20 iterations into our machine and that Rasqal has been able to detect and merge our three quantum calls together.

Here is what our code now looks like.

```
# This returns the 20 results directly in a list.
result = twenty_quantum_seeds()
```

By dynamically merging classical expressions into our quantum operations we've been able to provide a much larger and more fully detailed quantum circuit outlining what this hybrid code was trying to do. While this is not guaranteed to provide a better surface for optimization, as it may simply perform the same operation just in a larger circuit, but it provides far *more* context for optimizers. There is also the opportunity that later circuits may cancel out parts of earlier ones, which may not have been known without the full state provided by Rasqal.

But this is for future evolutions, as current machines almost universally perform better with smaller circuits. So unless merging generates a smaller circuit than each one individually, the benefits may only be fully realized in the future.

Because of this the current implementations of expression merging are limited, but the frameworks are available for both expansion and experimentation in the future.

2.4.6 Circuit Synthesis Module

Once a deferred result has been marked as requiring execution the projection it's linked to gets frozen and it starts building the circuit. The circuit building itself is also delayed until execution because expression merging and later state analysis additions may drastically change what gates are required. But once

execution is imminent, a projection collects what information it has available and turns the quantum operations into a circuit. It will also optimize at this time, but with a focus on optimizations that are universally applicable no matter what type of QPU is used. Aggressive niche optimizations are not applied here because at this point we are still unsure what sort of machine may run our circuit, and we rely upon a backends machine-specific optimizations to optimally run a circuit against its architecture. There is also the fact that different optimizers may eek different information out of a circuit, such as condensing certain patterns of gates, that prior optimization may inadvertently break the pattern for.

There is also a lot of potential for novel optimization and partial simulation techniques that can be applied during circuit synthesis due to the projections incremental build-up and view of full state. This is a critical point of future development.

Once a circuit has been built, including abstract lower-level instructions such as mid-circuit measurement and looping constructs, the projection queries available QPU backends for hardware requirements, such as features such as mid-circuit measurement availability, qubit count and allowed circuit depth. If a suitable backend is found the circuit is run against it, if not, an exception is thrown.

2.5 Python Bindings

While the majority of Rasqal is written in Rust it has Python bindings available and QPU integrations are also defined and built in Python as well. Since it's the quantum communities primary language it was an obvious choice to allow the widest away of integrations with both Rasqal itself but also with backends.

By default there is a Qiskit simulator and empty backend available, the latter primarily for testing. These can also be used as templates to add new backends.

3 Conclusion

In this paper we've introduced Rasqal, a classical-quantum hybrid runtime which symbolically executes hybrid IR in a dynamic and evolutionary fashion. S it holds a full program state at every step, it has unique opportunities for both execution and optimization of quantum algorithms that are unavailable to purely static, or even partially interpreted tools.

Included was a full breakdown of its architecture, primary data structures and key concepts. Included in this were details about its circuit synthesis module, graph evolution conditions, heavy deferred execution mechanism and how it deals with side-by-side mixed context, among others.

It's our hope that more dynamic systems with the potential for novel optimization and execution models will appear as the quantum compilation pipelines get more mature. Rasqal itself will continue its evolution, acting as a research platform for a variety of concepts that its constraints and runtime are best poised to exploit. Beyond bug fixes and finishing up currently partially complete components, all future work will be focused towards trying to exploit the dynamic execution nature and full contextual state available at each step of the symbolic executor.

3.1 Acknowledgments

3.2 References